

Feature Expansion With Semi-Dynamic Feature Sets (SDFS) and LLM-Powered Explainability: A Novel Approach to Discovering Missing Features

Elaheh Hosseini  and Soodeh Nikan , *Senior Member, IEEE*

Abstract—Traditional feature selection methods focus on reducing dimensionality, often at the cost of discarding valuable but hidden information. In contrast, this article introduces a feature expansion method, a novel approach that automatically identifies and reconstructs missing features, meaning attributes overlooked or undiscovered during initial feature extraction, enhancing predictive performance without requiring additional data collection. We propose the semi-dynamic feature set (SDFS) network, a unique machine learning framework that learns both feature values and model parameters, thereby improving robustness in complex datasets. To enhance interpretability, we integrate large language models (LLMs) to assign meaningful labels to the newly discovered features, transforming them from black-box variables into actionable insights. Our proposed feature expansion method consistently improved classification and regression performance across multiple datasets. In particular, the accuracy increased from 89% to 95% for Parkinson’s disease detection and from 82% to 85% for wine quality prediction, and reduced the mean squared error (MAE) in air quality prediction from 91.2 to 85.04, demonstrating its effectiveness in discovering missing features. By combining our feature expansion with LLM-driven explainability, this method not only enhances model’s performance, but also unveils hidden data patterns in critical domains such as healthcare, finance, and scientific research. The full implementation is available on GitHub [GitHub link], and the model can be installed via PyPI under the package “sdfs” [PyPI link].

Impact Statement—Feature engineering, particularly feature selection, is widely used to enhance machine learning performance and has been extensively studied. However, feature expansion as a way to transform the feature space of a given dataset into a richer and more interpretable representation to uncover previously hidden information is underexplored. In this article, we introduce a novel automated feature expansion method, emphasizing the interpretability and explainability of the newly discovered features. Our method significantly improves the predictive model’s performance while providing insights into aspects of the data that were missing during the initial feature extraction. This technique is particularly effective for small,

complex tabular datasets, whether tabular or time-series, and it may benefit critical areas such as healthcare, finance, and scientific research.

Index Terms—Explainability, feature engineering, feature expansion, large language models (LLMs), missing features.

I. INTRODUCTION

FEATURE selection has been a fundamental topic in machine learning, aiming to enhance model performance by selecting the most relevant features while reducing redundancy [1]. Popular feature selection algorithms such as recursive feature elimination (RFE) often use support vector machines (SVMs) or linear regression to identify key features, and principal component analysis (PCA) for dimensionality reduction in high-dimensional datasets. In addition, filter methods such as Chi-square and mutual information are commonly applied to text classification and bioinformatics for selecting features based on statistical relevance [2].

Conventional feature selection methods assume a complete feature space, but in real-world applications, such as chaotic systems or dynamic environments, critical features might be missing, leading to suboptimal model performance [8]. In many complex systems, such as weather forecasting or market analysis, the dynamics are usually intricate and often submerged, so they can only partially be captured in the early stages of feature extraction [3]. Some of these hidden features may not show any linear or nonlinear relationship with the existing extracted features [4]. Thus, they do not appear within the hidden layers in the neural network learning process. Therefore, overlooking influential features results in poor model convergence and provides suboptimal accuracy with reduced generalization capabilities [5].

In this article, we investigate a relatively underexplored concept in feature engineering known as feature expansion. In contrast to feature selection and dimensionality reduction, which aim to narrow down the feature space, feature expansion seeks to improve model performance by identifying potentially influential but previously unexplored features [6]. This is motivated by the fact that missing essential features within the input space can drastically reduce the precision of predictions, particularly in scenarios involving chaotic behavior or nonlinear dynamics [3].

Received 6 June 2025; revised 20 November 2025 and 13 February 2026; accepted 20 March 2026. Date of publication 7 April 2026; date of current version 31 August 2026. This article was recommended for publication by Associate Editor Ivor W. Tsang upon evaluation of the reviewers’ comments. (Corresponding author: Soodeh Nikan.)

The authors are with the Department of Electrical and Computer Engineering, Western University, London, ON N6A 3K7, Canada (e-mail: ehosse2@uwo.ca; snikan@uwo.ca).

Digital Object Identifier 10.1109/TAI.2026.3681539

The proposed semi-dynamic feature set (SDFS) model captures the missing features at the heart of our approach. Throughout the article, we will refer to two novel terms: static features and dynamic features. The former refers to the original features within a given dataset, and the latter refers to those potentially overlooked features that we aim to discover. The SDFS network enhances the feature space by dynamically estimating missing features throughout the training process, enabling it to capture hidden patterns that traditional models might overlook. Unlike most classical feature engineering methods, SDFS iteratively refines the expanded feature set, using backpropagation, to update the dynamic features along with model weights directly. This peculiar characteristic allows the model to learn those intricate dependencies that would otherwise remain hidden.

This novel approach addresses a critical gap in data science by managing features that were previously overlooked or not discovered during the initial stages of feature extraction within a certain dataset. This is particularly relevant in healthcare or finance applications where data can be limited, and every missing feature might result in a critically false outcome. By finding the estimated values of those missing features among existing samples, our approach can significantly reduce the need to collect more data, which is often expensive or impractical [7].

The main contributions of our study are outlined as follows.

- 1) *Introducing a Novel Feature Expansion Approach:* This article's main objective is to introduce a novel feature engineering approach that expands the initial input space into a higher dimension to find the possibly missing features in the input data through an innovative learning process. The proposed technique has great potential in many applications with complex and incomplete datasets.
- 2) *SDFS Network Structure:* We proposed two SDFS frameworks (Tabular-SDFS and Temporal-SDFS) designed for tabular datasets and time-series data, respectively. The Tabular-SDFS follows a structure similar to a simple MLP with a single hidden layer, while the Temporal-SDFS is implemented as an LSTM-based network. Both of these methods train a section of the input space, called dynamic features, along with the hidden layer's weights. These structures help to dynamically find the most proper values for the missing features within a training loop.
- 3) *Feature-Level Explainability:* We proposed a novel approach not only to find the values of dynamic features, but also to assign meaningful interpretations to the newly generated features. After finding the optimum number of dynamic features and estimating their values via the SDFS model, using a pre-trained large language model (LLM), our framework provides insights into their relevance and impact on model predictions. This provides insights into areas where the original feature extraction process may have failed to capture relevant information.

In what follows in this article, Section II explores existing research and describes our proposed approach's discrepancy versus current methods. Section III presents the technical details of the proposed SDFS and feature expansion models. It also describes the utilized datasets and their pre-processing steps. Section IV discusses the experimental evaluations the

explainability of the extended features. The article is concluded in Sections V and VI.

II. RELATED WORK

Feature selection has been one of the main areas of machine learning research, starting from the early work by Kohavi et al. [9] on enhancing model accuracy by reducing the complexity of feature space, which theoretically gave ground for choosing the most influential features while taking into account a trade-off between model accuracy and computational cost. Another seminal work by Dash and Liu [10] presented the consistency-based search in feature selection to ensure that selected features maintain consistency in predictions across various subsets of features. Later works, such as Hall's correlation-based feature selection, extended these ideas into addressing high-dimensional data problems and made feature selection considerably more efficient for complex datasets (L. Yu et al.) [11]. More recently, Moran and Gordon [12] introduced DCFS, a curiosity-driven reinforcement learning approach that selects informative features through intrinsic rewards.

Chaotic systems often need to revisit most of the assumptions about the availability of a full complement of features. Chaotic dynamics can be extremely sensitive to missing features in any system, where precise modeling is hard to execute. According to the article by Mitra et al. [13] on unsupervised feature selection, dynamic systems with missing variables can lead to poor convergence of models, hence leading to chaotic and unpredictable outputs. Moreover, in physics, the consequences of missing features on chaotic systems have been taken into consideration in great detail, where the behavior of dynamic systems can vary upon the omission of vital variables. Guyon et al. [14] provided state-of-the-art work by highlighting the importance of extracting those features that capture the crucial dynamics in perspective when dealing with complex systems.

Recent research has concentrated on the impact of missing variables in physics-based chaotic systems, explicitly studying how the absence of certain features in such systems would lead to unpredictable or unstable behavior. Kantarakias et al. [15] studied uncertainty quantification in chaotic systems with partially observed data. They developed the concept of polynomial chaos expansion (PCE) to overcome the sensitivity of chaotic systems to the lack of features, and its application in models such as the Lorenz system. More recently, another study by Zalp et al. [16] used machine learning methods to look at reconstruction and stability in chaotic systems where data is partial. They showed how incomplete observations result in the misrepresentation of system stability and further proposed recurrent neural networks to infer hidden dynamics [16]. Although computationally intensive, these methods provide a more accurate reconstruction of chaotic attractors while analyzing the system stability for incomplete vital features.

A related line of research on learnable feature expansion is presented by Sun et al. [17], who introduced feature expanded graph neural networks (FE-GNNs). Their method explicitly analyzes the feature space of GNNs and reveals that repeated neighborhood aggregations produce linearly correlated

TABLE I
CHARACTERISTICS OF DIFFERENT TABULAR DATASETS

Index	Dataset	Total Samples	Num of Features (Static)
1	Wine quality	6497	11
2	Pima indians diabetes	768	8
3	Breast cancer wisconsin	569	30
4	Census income	32 537	14
5	Parkinson disease detection	195	22
6	Diabetes health indicators	253 680	21

TABLE II
CHARACTERISTICS OF DIFFERENT TIME-SERIES DATASETS

Index	Dataset	Total Samples	Window Size	Num of Features (Static)
1	ETTh1	17 420	48	7
2	Air Quality UCI	6941	24	11

TABLE III
CLASS DISTRIBUTION OF SAMPLES IN EACH DATASET BEFORE SPLITTING

Index	Dataset	Num of Samples Class 0	Num of Samples Class 1
1	Wine quality	5117 (Not good quality)	1380 (Good quality)
2	Pima indians diabetes	268 (Diabetic)	500 (Non-diabetic)
3	Breast cancer wisconsin	212 (Malignant)	357 (Benign)
4	Census income	7840 (Income >50K)	24797 (Income $\leq$ 50K)
5	Parkinson disease detection	147 (Parkinson's negative)	48 (Parkinson's positive)
6	Diabetes health indicators	213 703 (Non-diabetes)	39 977 (Diabetes)

subspaces that limit representational diversity. That mechanism effectively expanded the learnable feature space and improved expressiveness and generalization. Radhakrishnan et al. [18] showed that neural networks learn new features by focusing on directions in the data that mostly affect their output. They called this process the average gradient outer product (AGOP), which enables a built-in feature learning ability in simpler models, such as kernel machines. Their results suggest that when a model expands its feature space by creating new and more informative versions of the original inputs, it can understand data patterns more deeply and perform better on complex tasks.

In addition, within the context of quantum physics, research on chaotic quantum systems demonstrates that missing information in the eigenvalue spectrum can introduce a turning point in the system dynamics. This observation was explained by J. Che et al. [19] on chaotic quantum systems with symplectic symmetry. The purpose of their work was to explore how missing eigenvalues can distort statistical properties and stability in complex quantum graphs.

All these studies indicate that the influence of missing features extends beyond simple machine learning problems to chaotic and quantum systems, where the absence of key features can change the essence of the behavior and predictability of the entire system [20]. However, the existing approaches primarily focus on reducing the input space into a less complex representation, or they do not address the explainability of their generated features. To address these gaps, we proposed a novel method to expand the feature space by adding new dimensions and training them to align with a more complex yet realistic representation. Furthermore, by utilizing LLMs, the potential

missing features can not only be estimated but also labeled, to enhance interpretability and provide meaningful insights into their significance.

III. MATERIALS AND METHODS

A. Datasets

In this study, six different tabular datasets along with two time-series datasets, have been utilized to evaluate our proposed feature expansion technique and draw a general conclusion about the types of data and problems where adopting SDFS as a preprocessing step would be effective. We provided a brief description of the tabular and time-series datasets in TABLE I and TABLE II, respectively.

B. Data Pre-Processing

In this article, we focus on two types of computational tasks, namely binary classification on tabular data and regression on time-series data. TABLE III shows the data distribution for each category of target values within each tabular datasets before splitting. For each dataset, we had an initial split of the dataset into three training, test, and validation subsets with 80% for training, and 10% allocated to each of the test set and validation sets. For the time-series datasets, order-preserving splitting strategy is applied to preserve the sequential nature of the data. However, the datasets had to go through the following specific pre-processing steps before data splitting:

1) *Wine Quality*: This dataset is a popular data for classification tasks that can be used for binary or multiclass classification

scenarios. Within this data set, multiple factors were used to predict the quality of different wine samples [21]. During the pre-processing steps, each column's missing values were filled with the respective column mean. Based on 11 variables, it classified wine quality into one of 11 categories between 0 and 10. This dataset also needed to have its target variable binarized. All features were also transformed and were constrained to have a zero mean and a variance of 1. The dataset consists of 6497 samples. We reserved 5262 instances for training, 650 for the testing phase, and 585 for validation.

2) *Pima Indians Diabetes*: The purpose of this dataset is to predict whether or not a patient has diabetes based on certain features, such as blood pressure and skin thickness [22]. As the initial pre-processing steps on this dataset, missing values were replaced by their respective average values. This dataset has 768 samples, where 621 samples were held for training, and 77 and 70 samples for the test and validation sets, respectively.

3) *Breast Cancer Wisconsin*: This dataset uses cell nuclei presented in the images to extract features from a breast mass of a fine needle aspirate (FNA), which can be utilized to predict breast cancer in patients [23]. The only preprocessing step applied to this dataset was the removal of the 'id' column. After this, each sample retained 30 features, along with a target column indicating whether the tumor was malignant or benign. This dataset has 569 samples and, after splitting them into three separate subsets, we had 460, 57, and 52 samples within the training, test, and validation sets, respectively.

4) *Census Income*: The prediction task in this dataset is to determine whether a person makes over \$50 K annually, using factors such as relationships, education, and race [24]. In this dataset, missing values within this dataset were replaced with the most frequently occurring value in the corresponding column. This dataset contains 32 537 samples in general, and after splitting, we had 26 354, 3254, and 2929 samples within the training, test, and validation sets, respectively.

5) *Parkinson's Disease Detection*: This dataset is used to predict Parkinson's disease using vocal features and nonlinear signal processing measures as input attributes [25]. To preprocess this dataset, we filled the missing values with the average of the corresponding column. This dataset has 195 samples in total, and after splitting, we had 157, 20, and 18 samples within the training, test, and validation sets, in that order.

6) *Diabetes Health Indicators*: This dataset was collected from the CDC behavioral risk factor surveillance system (BRFSS) to predict whether a person has diabetes based on 21 health and lifestyle indicators, such as body mass index (BMI), smoking habits, physical activity, and general health conditions, [26]. During preprocessing, the original target column (Diabetes_012) was converted into a binary label by merging classes 1 and 2 as a single positive category, renamed as Diabetes_binary. To handle extreme values, the BMI column was adjusted using the interquartile range (IQR) method through winsorization. After cleaning and processing, the dataset contained 253 680 samples, which were split into 205 480 for training, 22 832 for validation, and 25 368 for testing.

7) *ETTh1*: The ETTh1 dataset contains hourly temperature and energy-related features collected from an electric

transformer station [27]. It includes seven variables such as high- and low-use load, and outdoor temperature (OT). As pre-processing, the records were sorted chronologically based on their timestamps after converting the date column to datetime format. A sliding window of 48 h was used to segment the time series into input sequences. In total, we had 17 420 samples, with 13 888 windows used for training, 1694 for validation, and 1694 for testing.

8) *Air Quality UCI*: This time-series dataset contains hourly measurements of different air pollutants and meteorological variables collected in an Italian city [28]. The target variable in this dataset is $NO_x(GT)$, representing the concentration of nitrogen dioxide. During preprocessing, the Date and Time columns were merged into a single timestamp column and sorted chronologically. Missing or invalid values (marked as -200) were removed, and only the essential features and target column were kept. After cleaning, 6941 valid samples remained. Using a window size of 24, the data were split into training, validation, and test sets with sizes of 5262, 650, and 1029 windows, respectively.

C. Tabular-SDFS

1) *Dynamic Feature Initialization*: As a first step for any dataset in our study, we begin by introducing an imaginary set of impactful features that may not exist in the original input space. We refer to this new feature set as dynamic features, as they are designed to be updated and modified throughout the training loop over time. This contrasts with the original features in the datasets, which are referred as static features and remain unchanged throughout the entire analysis. The number of dynamic features is determined through a brute-force search, and we select the optimal number by comparing the final results across different datasets separately. Then, we need to initialize the dynamic feature values for each sample in the training set. Regular hidden weights tend to be randomly initialized. However, since the initialized values of a trainable set can significantly impact both the convergence speed and the final calculated trained set, we preferred to explore initialization methods rather than random initialization. In general, we have tried three different approaches for initializing the dynamic features for training samples in tabular datasets. First, we used random initialization as a baseline approach. Second, we initialized the dynamic features based on the mean and standard deviation of the training set samples. Finally, we applied principal component analysis (PCA) to reduce the dimensionality of the input space to a fixed number of components, corresponding to the intended size of the dynamic features.

2) *Updating Dynamic Features*: After the initialization step of the dynamic features, the static features, alongside the initialized dynamic features, needed to be fed to the SDFS network. The Tabular-SDFS model is an MLP with one hidden layer of 128 neurons. Unlike a standard MLP, however, the dynamic feature set as a part of the input features is also learnable during training, in addition to the usual weights and biases. The structure of this model is shown in Fig. 1. As illustrated in the figure, we adopted the PCA-based initialization technique in our final

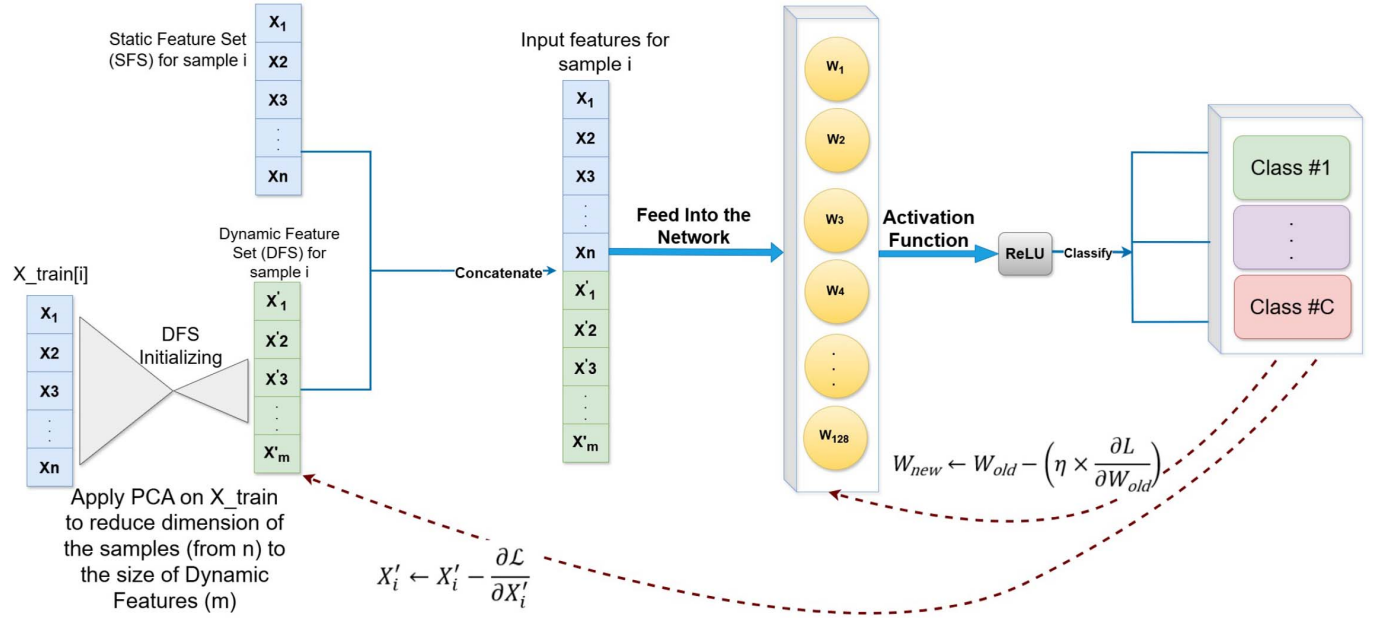


Fig. 1. Dynamic feature set updating path in an Tabular-SDFS network.

proposed structure, as it resulted in the fastest convergence in the training phase of the SDFS model, as well as the highest performance in most cases, in the evaluation phase.

During training the SDFS model, the corresponding dynamic feature set of i th sample, X'_i , was updated directly from the gradients by using

$$X'_i \leftarrow X'_i - \frac{\partial \mathcal{L}}{\partial X'_i}. \quad (1)$$

For all other weights of the hidden layer, the network calculates the updated weights as follows:

$$W_{\text{new}} \leftarrow W_{\text{old}} - \left(\eta \times \frac{\partial \mathcal{L}}{\partial W_{\text{old}}} \right). \quad (2)$$

Note that in (1) and (2), $\mathcal{L}$ represents the gradient, and η denotes the learning rate.

As shown in Fig. 1, to update the dynamic features, we kept the learning rate constant at 1 and applied the total gradient directly. This approach enables faster tuning of the dynamic features for each sample by mitigating the smoothing effect of the learning rate, leading to rapid convergence and a more precise fit to the data. In addition, it was experimentally found that using a learning rate of 1 to update dynamic features not only leads to faster convergence, but also results in higher test and validation accuracy. The impact of this design choice is evaluated through a brief ablation study in Section IV-A.

Despite the positive effects of a constant learning rate when updating the dynamic features, this approach significantly increases the risk of overfitting. To address this issue, we employed two different strategies, which will be discussed in the following section.

A notable characteristic of the SDFS model is that during training, dynamic features must be updated individually for each sample. As a result, we had to train the model stochastically without the ability to adjust batch size or use batch processing. Although this particular aspect of the SDFS model may have negative impacts on the speed of network training, it is a fundamental aspect of the model that must be carefully accounted for.

3) *Over-Fitting Prevention Techniques:* We proposed an SDFS network to be as simple as possible to reduce the risk of overfitting. However, during the training of the SDFS model and evaluation on the test samples, a significant difference was observed between training and validation accuracy. The training accuracy quickly increased to almost 100% after only a few epochs (often in less than ten epochs). However, validation accuracy did not improve as much, suggesting that the network was likely overfitting to the training data. Fig. 2 illustrates the loss and accuracy curves for the training and validation sets on the wine quality dataset. As noted earlier, we set the learning rate to 1 when updating the dynamic features, effectively applying the full gradient to accelerate the process without compromising the final accuracy. This aggressive update strategy is likely a key factor contributing to the early appearance of overfitting after only four epochs. To tackle this problem, we employed two approaches, as follows:

a) *L2 regularization (weight decay):* To avert the model from relying too heavily on specific training samples, we applied L2 regularization, also known as weight decay, inside the training loop of the SDFS network. This method adds a penalty parameter to the loss function which limits the magnitude of the weights. By keeping the weights smaller, we reduce the risk of overfitting. In this study, we set the regularization parameter ($\lambda = 1e^{-5}$), which strikes a balance between regularizing the model and allowing it to be more generalizable. We modified

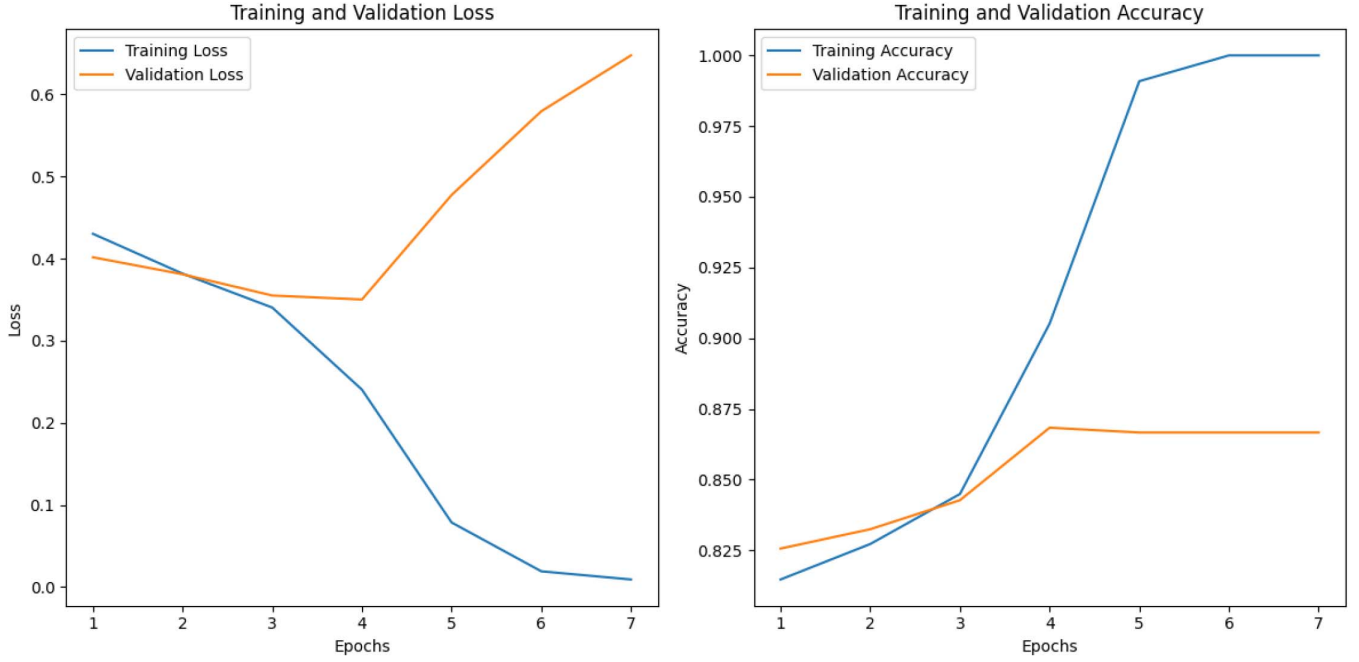


Fig. 2. Training and validation loss and accuracy plots for the wine quality dataset.

the loss function to include the weight decay term as follows:

$$L_{\text{total}} = L_{\text{model}} + \lambda \sum w_i^2 \quad (3)$$

where L_{model} is the original loss (e.g., cross-entropy), w_i represents the model weights, λ is the regularization parameter, and L_{total} is the final applied loss.

It is also important to note that this regularization technique only applies to hidden-layer weights and has no direct impact on the dynamic feature update process. However, since it influences the latent space by altering the weights of the hidden layer, it will also have an indirect effect on the dynamic features as well.

b) Early stopping: As a second approach to address the overfitting problem, we employed an early stopping technique, where the performance of the model on the validation set is monitored during the training steps. If the validation loss does not improve after a certain number of epochs, we stop the training phase. For our experiments and due to the fast convergence of our process, we set this threshold (patience) to 3 epochs. Early stopping ensures that the model does not overtrain on the samples and does not learn noise, which helps to improve its performance on unseen test data.

The pseudocode of the entire process discussed above, is shown in Algorithm 1. This pseudocode outlines the details of our proposed Tabular-SDFS network structure along with its training loop.

4) Expanding Test-Set Samples: After training the SDFS network and updating the dynamic feature set of each training samples individually, it is also necessary to apply an expansion technique to the test and validation sets, to equalize the shape of

Algorithm 1: Tabular-SDFS Dynamic Feature Update for-Training Samples.

Input: Training set $X = \{(X_i, y_i)\}_{i=1}^N$, Number of dynamic features m , Learning rate η for weights

Output: Updated dynamic features X'_i

- 1 Initialize dynamic feature set X'_i for each X_i using PCA;
 - 2 Initialize model weights W and biases b ;
 - 3 **repeat**
 - 4 **foreach** training sample (X_i, y_i) **do**
 - 5 Concatenate $Z_i \leftarrow [X_i \parallel X'_i]$;
 - 6 Forward propagate Z_i through the MLP to get prediction $\hat{y}_i$;
 - 7 Compute loss $L_i \leftarrow \text{Loss}(\hat{y}_i, y_i)$;
 - 8 Compute gradient $\nabla_{X'_i} L_i$ and $\nabla_W L_i$;
 - 9 Update dynamic features: $X'_i \leftarrow X'_i - \nabla_{X'_i} L_i$
 - 10 Update weights: $W \leftarrow W - \eta \cdot \nabla_W L_i$;
 - 11 Apply L2 regularization: $L_{\text{total}} \leftarrow L + \lambda \sum W^2$;
 - 12 Monitor validation loss for early stopping;
 - 13 **until**;
 - 14 Validation loss stops improving (patience = 3 epochs)
-

all samples across different subsets. This step is crucial to prevent data leakage by ensuring that the test and validation sets remain completely hidden from the SDFS network throughout the training process. To tackle this, first, for each test or validation sample, the closest data point within the training set was found by comparing only their static features. The trained dynamic features of the closest training sample were then associated with that particular test or validation sample.

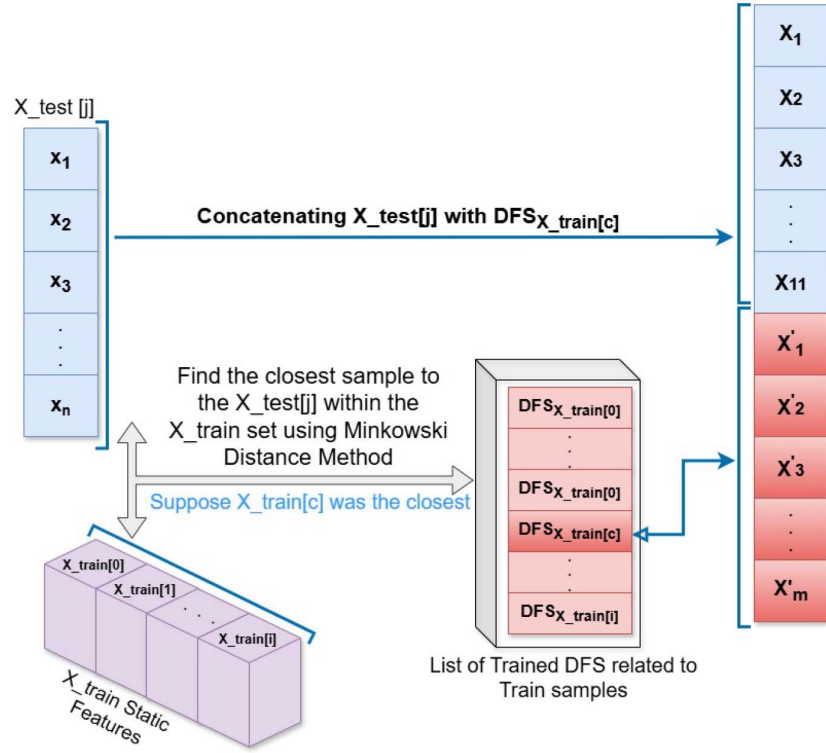


Fig. 3. Concatenating dynamic feature set to static features of each test sample.

For this step, we experimented with multiple distance-metrics, including Euclidean, Manhattan, Minkowski, and Cosine distances. In most cases, using Minkowski distance resulted in the highest test accuracy across different datasets. The feature expansion process for the test samples is illustrated in Fig. 3.

We also applied a similar approach to the validation set during each epoch of the training loop, assigning a corresponding set of dynamic features to each validation sample after every training iteration, while ensuring they remained completely hidden from the SDFS network.

To clarify this process, we provided a separate pseudocode focused on the feature expansion process for the test and validation samples, as shown in Algorithm 2.

D. Temporal-SDFS

Similar to the SDFS pipeline designed for tabular datasets, we developed an LSTM-based SDFS architecture customized for sequential data. As illustrated in Algorithm 3, the training procedure closely follows the structure presented for tabular data in Algorithm 1, except that the core SDFS network is implemented using an LSTM model and is designed for regression tasks. In this framework, the dynamic features are initialized randomly before training and are iteratively updated during the optimization process. However, the strategy for expanding the validation and test subsets differs significantly, as discussed in the following section.

1) *Closest Trend Identification in Temporal-SDFS*: As described in Section III-C, Tabular-SDFS, the test and validation

Algorithm 2: Assign Dynamic Features to Test/Validation Samples in Tabular-SDFS.

Input: Trained dynamic feature sets $\{X'_{\text{train}[i]}\}_{i=1}^N$,

Training static features $\{X_{\text{train}[i]}\}_{i=1}^N$,

Test/Val static features $\{X_j\}_{j=1}^M$

Output: Extended test/val inputs $\{Z_j = X_j \parallel X'_{\text{assigned}[j]}\}$

```

1 foreach test/validation sample  $X_j$  do
2    $\text{minDistance} \leftarrow \infty$ ;
3    $\text{nearestIndex} \leftarrow -1$ ;
4   foreach training sample  $X_{\text{train}[i]}$  do
5     Compute distance  $d \leftarrow \text{Minkowski}(X_j, X_{\text{train}[i]})$ ;
6     if  $d < \text{minDistance}$  then
7        $\text{minDistance} \leftarrow d$ ;
8        $\text{nearestIndex} \leftarrow i$ ;
9   Assign  $X'_{\text{assigned}[j]} \leftarrow X'_{\text{train}[\text{nearestIndex}]}$ ;
10  Concatenate  $Z_j \leftarrow [X_j \parallel X'_{\text{assigned}[j]}]$ ;

```

subsets were expanded without directly exposing them to the SDFS pipeline. This was achieved by identifying the closest sample in the training subset, for each test or validation sample, solely based on their static features, and then reusing the corresponding trained dynamic features. When dealing with time-series data, in addition to maintaining the same requirement of keeping the test and validation windows unseen by the SDFS

Algorithm 3: Temporal-SDFS-Dynamic Feature Update with LSTM (Regression).

```

1
Input: Training windows  $\{(X_i^{\text{stat}}, y_i)\}_{i=1}^N$ , window size  $W$ ;
    Window size  $W$ ;
    Number of dynamic features  $m$ ;
    Learning rate  $\eta$  for model;
    Weight decay  $\lambda$ ;
    Dropout  $p = 0.1$ ;
    Early-stopping patience  $P = 5$ .
Output: Updated dynamic features  $\{D_i\}_{i=1}^N$ .
2 Model:  $f_\theta = \text{Linear}(\text{Dropout}(p)(\text{LSTM}(\cdot)))$ ;
3 Initialize  $D_i$  For each window  $i$ , with shape  $(W, m)$ 
  (random init).;
4 Initialize network parameters  $\theta$ ;
5  $L_{\text{best}} \leftarrow \infty$ ,  $c_{\text{pat}} \leftarrow 0$ ;
6 repeat
7   foreach training window  $(X_i^{\text{static\_features}}, y_i)$  do
8      $Z_i \leftarrow [X_i^{\text{static\_features}} \parallel D_i]$ ;
      // concatenate along feature
      dimension at each time step
9      $\hat{y}_i \leftarrow f_\theta(Z_i)$ ; // forward through LSTM
       $\rightarrow \text{Dropout}(0.1) \rightarrow \text{Linear layer}$ 
10     $L_i \leftarrow \ell(\hat{y}_i, y_i) + \lambda \|\theta\|_2^2$ ; // regression
      loss (e.g., MSE) + L2
      regularization
11    Compute  $\nabla_{D_i} L_i$  and  $\nabla_\theta L_i$  by
      backpropagation.;
12     $D_i \leftarrow D_i - \alpha \nabla_{D_i} L_i$ ; // manual update
      of dynamic features
13     $\theta \leftarrow \theta - \eta \nabla_\theta L_i$ ; // optimizer step
      for LSTM parameters
14    Compute validation loss  $L_{\text{val}}$  with  $\theta$  and current
       $\{D_i\}$ ;
15    Monitor  $L_{\text{val}}$  for early stopping.;
16 until early stopping is not triggered;
17 return  $\{D_i\}_{i=1}^N$ ;

```

network, it is necessary to consider the temporal dependency between samples. To address this, we developed a method that, instead of finding the closest individual sample, identifies the most similar trend or window within the training set for each test or validation window. In other words, for each validation or test window X_j , the objective is to assign an appropriate dynamic-feature set belonging to a training window that reflects the most similar temporal behavior to X_j . To achieve this, the method searches for the closest trend among all training windows $\{X_{\text{train}}[i]\}_{i=1}^N$ based on normalized shape similarity.

To find this similarity, each training and test or validation window is independently z-normalized along the temporal dimension to remove scale and offset biases based on the

following equations:

$$\tilde{X}_{\text{train}}[i] = \frac{X_{\text{train}}[i] - \mu(X_{\text{train}}[i])}{\sigma(X_{\text{train}}[i])} \quad (4)$$

$$\tilde{X}_j = \frac{X_j - \mu(X_j)}{\sigma(X_j)} \quad (5)$$

where $\mu(\cdot)$ and $\sigma(\cdot)$ denote the mean and standard deviation computed per feature over time.

Each normalized window is then flattened into a vector, and a cosine similarity score is computed between $\tilde{X}_j$ and $\tilde{X}_{\text{train}}[i]$

$$\text{sim}(i, j) = \frac{\tilde{X}_{\text{train}}[i]^\top \tilde{X}_j}{\|\tilde{X}_{\text{train}}[i]\|_2 \|\tilde{X}_j\|_2}. \quad (6)$$

Equation (6), measures the alignment of temporal patterns independently of their absolute magnitudes. After that, the training window with the highest similarity score is selected as the most shape-consistent trend using the following equation:

$$i^* = \arg \max_i \text{sim}(i, j). \quad (7)$$

To preserve amplitude relations, a proportional scaling factor is computed from the mean-centered versions of both windows

$$s = \frac{\|X_j - \mu(X_j)\|_F}{\|X_{\text{train}}[i^*] - \mu(X_{\text{train}}[i^*])\|_F} \quad (8)$$

where $\|\cdot\|_F$ is the Frobenius norm.

The dynamic-feature matrix of the matched training window is scaled by s and assigned to the test or validation window

$$\hat{D}_j = s \cdot D_{\text{train}[i^*]}^*. \quad (9)$$

The final extended representation for the test or validation window is obtained by concatenation as formulated in (10) below

$$Z_j = [X_j \parallel \hat{D}_j]. \quad (10)$$

By reusing the corresponding dynamic features (appropriately scaled), the model can maintain temporal coherence and structural consistency in downstream predictions without re-training on validation or test data. The detailed pseudocode describing this procedure is provided in Algorithm 4.

IV. RESULTS

A. Tabular-SDFS Performance

In this study, we used multiple approaches to comprehensively evaluate our proposed feature expansion technique. As depicted in TABLE I, a variety of datasets were utilized to have a better assessment.

First, we utilized a neural network similar to the Tabular-SDFS structure on each dataset as a baseline model. However, the feature space was not expanded on the training samples. After training this simple model with only a single hidden layer of 128 neurons, we evaluated that model on the test samples for each dataset. The results of multiple evaluation metrics are shown in Table IV.

Subsequently, we tested the performance of the Tabular-SDFS model as a preprocessing step and the impact of the

Algorithm 4: Assigning Dynamic Features to Val/Test Windows via Shape Matching and Scaling.

```

1
Input: Trained dynamic features  $\{D_{\text{train}[i]}^*\}_{i=1}^N$  with
        shape  $(W \times m)$ ;
        Training windows  $\{X_{\text{train}[i]}\}_{i=1}^N$  with shape
         $(W \times F_s)$ ;
        Val/Test windows  $\{X_j\}_{j=1}^M$  with shape  $(W \times F_s)$ ;
        small  $\varepsilon > 0$ .
Output: Extended val/test windows  $\{Z_j\}_{j=1}^M$  where
         $Z_j = [X_j \parallel \hat{D}_j]$ .

2
3 foreach val/test window  $X_j$  do
4    $y \leftarrow X_j$ ;  $y_z \leftarrow \text{z\_norm\_along\_time}(y)$ ;
    $y \leftarrow \text{vec}(y_z)$ ;
5    $\text{bestSim} \leftarrow -\infty$ ;  $\text{bestIdx} \leftarrow -1$ ;
6   foreach training window  $X_{\text{train}[i]}$  do
7      $x \leftarrow X_{\text{train}[i]}$ ;
      $x_z \leftarrow \text{z\_norm\_along\_time}(x)$ ;
      $x \leftarrow \text{vec}(x_z)$ ;
8      $\text{sim} \leftarrow \frac{x^\top y}{(\|x\|_2 + \varepsilon)(\|y\|_2 + \varepsilon)}$ ; // cosine
     similarity of z-normalised
     shapes
9     if  $\text{sim} > \text{bestSim}$  then
10       $\text{bestSim} \leftarrow \text{sim}$ ;  $\text{bestIdx} \leftarrow i$ ;
11    $X^* \leftarrow X_{\text{train}[\text{bestIdx}]}$ ;  $Y^* \leftarrow X_j$ ;
12    $\mu_x \leftarrow \text{mean along time}(X^*)$ ;
    $\mu_y \leftarrow \text{mean along time}(Y^*)$ ;
13    $X_c \leftarrow X^* - \mu_x$ ;  $Y_c \leftarrow Y^* - \mu_y$ ;
14    $\text{scale} \leftarrow \frac{\|Y_c\|_F}{\|X_c\|_F + \varepsilon}$ ; // scale from
   centered Frobenius norms
15    $\hat{D}_j \leftarrow \text{scale} \cdot D_{\text{train}[\text{bestIdx}]}$ ; // scaled
   dynamic features from closest
   train window
16    $Z_j \leftarrow [X_j \parallel \hat{D}_j]$ ;
17 return  $\{Z_j\}_{j=1}^M$ ;

```

extended dynamic features on the evaluation metrics for each dataset. First, the optimal number of dynamic features for each dataset was found by varying the number of missing features between 1 and 10. Then the performance of the SDFS network with that optimal number of missing features was evaluated on the test samples. Table IV shows the optimal number of missing features for each dataset in the fourth column. The best initialization and distance methods are shown in columns 3 and 6, respectively, according to the performance of the final classification model on the test samples, which is again a simple MLP with a single hidden layer of 128 neurons, similar to our baseline model.

Table VI presents an assessment of the SDFS network performance on all utilized tabular datasets. As shown in the table,

in most cases, by using our proposed feature expansion method, the performance of the applied model improved on all metrics.

To further examine the effect of the learning rate used for updating dynamic features, we conducted a brief ablation study on the Wine Quality dataset by varying the learning rate across $\{1, 0.1, 0.01, 0.001\}$. The results are summarized in Table VII. As observed, reducing the learning rate does not improve predictive performance and substantially increases the number of epochs required for convergence. These findings support the choice of using a learning rate of 1 in the proposed framework.

In the evaluation of the proposed model, we extended the analysis beyond a simple comparison with the baseline model by including two well-known feature expansion techniques, which are polynomial feature expansion and autoencoder-based feature expansion. Polynomial feature expansion [29] augments input features by generating interaction terms up to a specified degree, enabling the model to capture nonlinear relationships. Autoencoder-based expansion [30] uses a neural network to learn compact, informative representations, which are then decoded to generate expanded features. We particularly included the autoencoder to allow for a fair comparison between our learning-based feature expansion method and another learning-based feature extraction approach. The results of these comparisons on the Wine Quality dataset, as our primary case study, are presented in Table VIII. Although Table VIII focuses on this main dataset, it is worth noting that we also observed similar performance trends across all other datasets.

1) *Feature explainability:* Although our proposed method significantly improved performance across tabular datasets, it is crucial to assess whether feature expansion introduces any biases and also verify that the newly discovered dynamic features contribute meaningfully to the classification decisions. We used Shapley additive explanations (SHAP) analysis to interpret the output of our model. SHAP gives preliminary insights into the importance of dynamic features in determining the final output, based on the contribution of each dynamic feature at different stages during model training and prediction. Moreover, it gives insight into the relationship between the probability of a particular class and the values assigned to each of these dynamic features. This method was implemented in the wine quality dataset as a case study, and the plot of the corresponding SHAP values is shown in Fig. 4.

As noted in Table I, the wine quality dataset contains 11 attributes at the beginning, which are labeled as static features, indicating that these attributes are fixed and have been present in the dataset since its creation. As shown in Table V, the best number of dynamic features for this dataset is 4, with PCA as the optimal method to initiate these extended features and the Minkowski algorithm as the distance metric to find the nearest training sample to each validation or test sample in forming those extended subsets. Consequently, the extended version of this dataset contains 15 features (11 static and 4 dynamic features) for each instance in the training, test, and validation sets. All these 15 features are shown in the left sidebar in the SHAP plot in Fig. 4. The order of the features in this plot indicates their relative importance. As shown in this figure, *dynamic_feature_2* and *dynamic_feature_3* are the most essential

TABLE IV
MODEL PERFORMANCE ON DIFFERENT DATASETS WITHOUT UTILIZING FEATURE EXPANSION

Index	Dataset	Accuracy	Recall	Precision	F1-Score	ROC-AUC
1	Wine Quality	0.82	0.61	0.71	0.63	0.83
2	Pima Indians Diabetes	0.72	0.70	0.70	0.70	0.80
3	Breast Cancer Wisconsin	0.97	0.97	0.97	0.97	0.99
4	Census Income	0.82	0.69	0.77	0.72	0.85
5	Parkinson Disease Detection	0.89	0.71	0.94	0.77	0.92
6	Diabetes Health Indicators	0.86	0.50	0.43	0.46	0.29

TABLE V
OPTIMAL SDFS HYPER-PARAMETERS AND CHARACTERISTICS FOR EACH DATASET

Index	Dataset	Initializing Method	Num of DF	SDFS Convergence Time (s)	Distance Method
1	Wine Quality	PCA	4	35	Minkowski
2	Pima Indians Diabetes	PCA	2	4	Minkowski
3	Breast Cancer Wisconsin	Mean-Std	6	7	Minkowski
4	Census Income	Randomly	2	210	Manhattan
5	Parkinson Disease Detection	PCA	2	4	Minkowski
6	Diabetes Health Indicators	PCA	4	1500	Minkowski

TABLE VI
BEST PERFORMANCE OF SDFS NETWORK ON EACH DATASET

Index	Dataset	Accuracy	Recall	Precision	F1-Score	ROC-AUC
1	Wine Quality	0.85	0.77	0.76	0.77	0.87
2	Pima Indians Diabetes	0.74	0.70	0.71	0.70	0.79
3	Breast Cancer Wisconsin	0.96	0.96	0.96	0.96	0.98
4	Census Income	0.83	0.71	0.77	0.73	0.84
5	Parkinson Disease Detection	0.95	0.96	0.88	0.92	0.99
6	Diabetes Health Indicators	0.86	0.55	0.65	0.57	0.79

TABLE VII
EFFECT OF THE LEARNING RATE USED FOR UPDATING DYNAMIC FEATURES ON TABULAR-SDFS CONVERGENCE AND PERFORMANCE ON THE WINE QUALITY DATASET

Learning Rate (η)	Test Accuracy	Convergence Epochs
1.0	0.85	7
0.1	0.84	13
0.01	0.83	26
0.001	0.83	30

Note: Bold indicates the summarized comparisons.

features in the prediction of the classification model. Meanwhile, dynamic_feature_4 and dynamic_feature_1, although very important, rank fourth and sixth, respectively. This shows that the extended dynamic features are effective in boosting the model's performance. As manifested in TABLE VI, it also gives a clear visualization of their crucial role in affecting the prediction of the model.

Furthermore, based on Fig. 4, the values of dynamic_feature_2, which is also identified as the most important feature in the classification process, are highly correlated with the probability of a sample being classified into class 1. This would imply that as the value of dynamic_feature_2 increases, the likelihood of the sample being classified as a high-quality wine decreases.

One of the key distinctions of this research compared with the prior work is that we went beyond merely identifying potentially missing features within the original input space. To

further enhance the interpretability of the dynamic features, we employed a large language model (LLM) to assign meaningful labels or descriptions to each feature. We provide a brief description of the wine quality dataset, including the names of all 11 static features, an explanation of how the dynamic features were generated, and a subsample of 100 instances with their corresponding 15 feature values (both static and dynamic). In addition, the mean value of each feature within the subset, the statistical correlations between dynamic and static features, and the results of the SHAP analysis are presented. These results highlight the direction and magnitude of the average contribution of each feature to the model's predictions for class 1 (high-quality wine samples). Using the above information and using the GPT-o1-preview-2024-09-12 model, one of the most powerful models released by OpenAI, we developed an innovative methodology to assign meaningful names to each of the four dynamic features specific to this dataset, as shown in Table IX. This novel application of LLMs highlights its exceptional capability to address complex, reasoning-intensive problems, which provides deeper insights into the underlying data. Using this technique, we not only made informed guesses about the dynamic features but also proposed a clear rationale behind each naming choice. The prompt used for this purpose and a summary of the LLM's response are presented in Fig. 5.

To evaluate the quality of the labels assigned to the dynamic features, we additionally reported the semantic alignment

TABLE VIII
PERFORMANCE COMPARISON OF THE PROPOSED SDFS METHOD AGAINST BASELINE AND
OTHER FEATURE EXPANSION TECHNIQUES ON THE WINE QUALITY DATASET

Method	Accuracy	Recall	Precision	F1-Score	ROC-AUC
Baseline	0.82	0.61	0.71	0.63	0.83
SDFS	0.85	0.77	0.76	0.77	0.87
Polynomial Expansion	0.83	0.64	0.74	0.67	0.86
Autoencoder	0.81	0.61	0.70	0.63	0.77

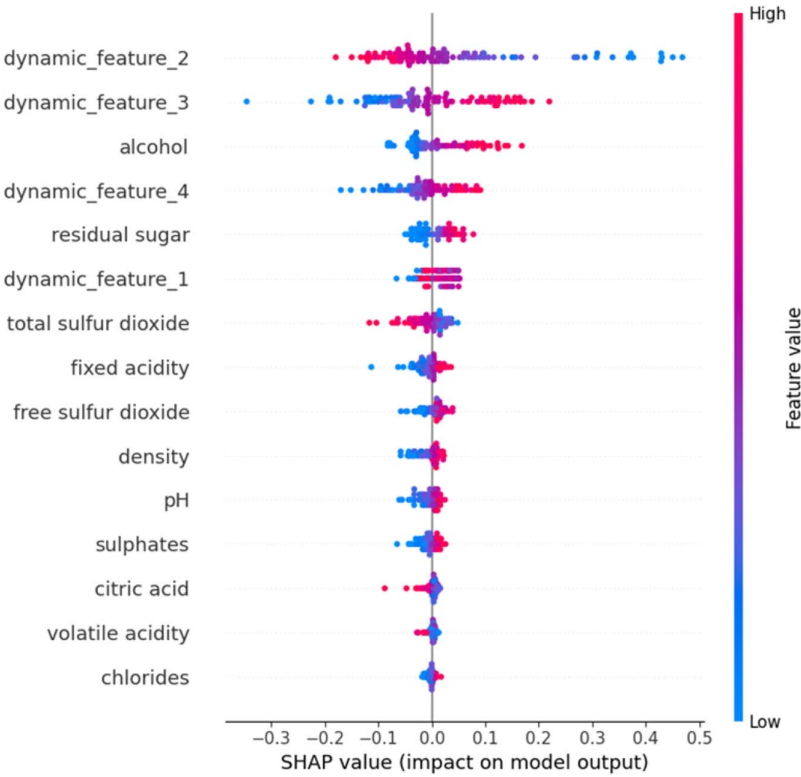


Fig. 4. SHAP values regarding class 1 (high-quality wines) within the wine quality dataset.

TABLE IX
DYNAMIC FEATURE ASSIGNED NAMES AND THE RATIONALE BEHIND
THEIR SELECTION BY LLM

Dynamic Feature 1	Assigned Name	“microbial stability and sweetness synergy index”
	Rationale	Reflects the combined influence of sulfur dioxide on microbial stability and residual sugar’s sweetness, capturing the balance between preserving sweetness and ensuring microbiological control.
Dynamic Feature 2	Assigned Name	“Non-alcoholic body and mineral content index”
	Rationale	Emphasizes the feature’s focus on the wine’s non-alcoholic components (density, acidity, sugars) and mineral-like chlorides, which together shape the wine’s body and mouthfeel without depending heavily on alcohol.
Dynamic Feature 3	Assigned Name	“Acidity structure and freshness index”
	Rationale	Highlights the core role in shaping the wine’s acidity profile, through fixed and citric acids, lower pH, and reduced volatile acidity, resulting in a fresh, vibrant wine structure.
Dynamic Feature 4	Assigned Name	“Oxidative stability and mouthfeel balance index”
	Rationale	Reflects feature’s emphasis on factors (sulphates, pH, free sulfur dioxide) that improve oxidative stability and enhance the wine’s texture and overall sensory balance.

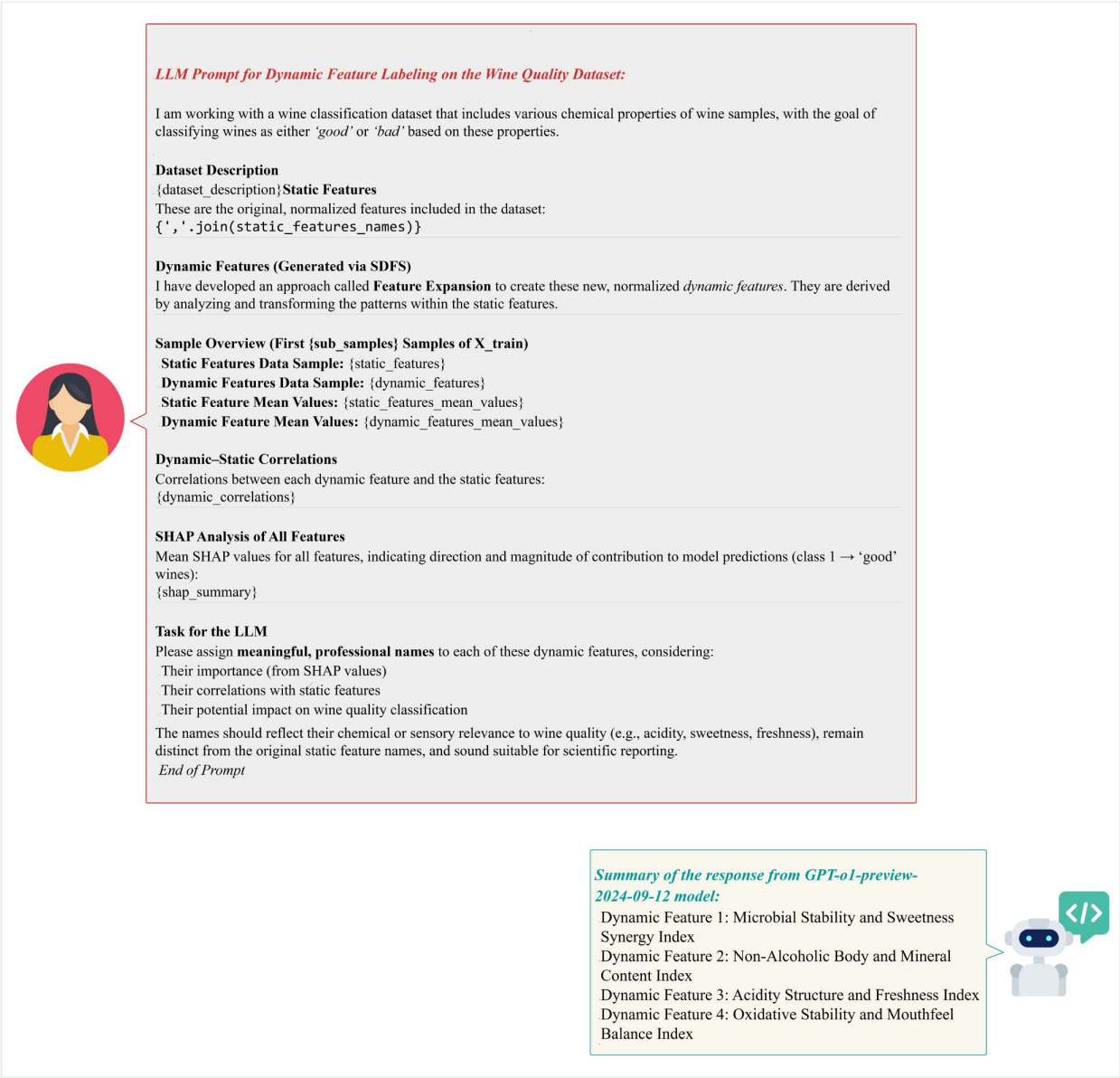


Fig. 5. Summarized interaction with GPT-o1-preview-2024-09-12 model for LLM-based dynamic feature labeling on the wine quality dataset.

scores across three different model types: RoBERTa-large-MNLI (moderately strict), DeBERTa-Xlarge-MNLI (highly conservative), and E5-large-V2 (sentence-level semantic similarity). To perform this evaluation, we first generated reference descriptions for each dynamic feature using a combination of SHAP value analysis and feature-target correlation scores. We then assessed the quality of the assigned labels by measuring their semantic similarity to these reference labels. These results are presented in Table X. Our results show that all generated labels maintain strong semantic relationships with their reference explanations, achieving E5 cosine scores above 0.81 and BERTScore F1 values up to 0.77.

2) *Generalization and Overfitting Analysis:* After evaluating the overall performance of Tabular-SDFS as a preprocessing method across various datasets and examining possible labels

or explanations for the newly generated dynamic features, we also need to ensure that the strong performance of our proposed model is not tied to a specific data split and can generalize well without significant loss in capability.

Another reason for this further investigation is to ensure that our SDFS preprocessing does not cause any overfitting in the final classifier model and that the classifier network remains stable and reliable.

To address this, we employed two different strategies, tested on our case study dataset, wine quality. First, we examined the model’s stability across random data splits. We used five different random seeds with the same 80/10/10 ratio for training, validation, and test set, respectively, and reported the mean $\pm$ standard deviation of Accuracy, F1-score, and ROC-AUC on the test set. These results were obtained using our baseline

TABLE X
EVALUATION OF SEMANTIC ALIGNMENT BETWEEN THE ASSIGNED FEATURE LABELS FOR EACH DYNAMIC FEATURE AND THEIR CORRESPONDING SHAP AND CORRELATION-BASED DESCRIPTIONS BASED ON THE WINE QUALITY DATASET

Feature	Assigned Label	SHAP and Correlation-Based Description	BERTScore F1 [roberta-large-mnli]	BERTScore F1 [deberta-xlarge-mnli]	E5 Cosine [e5-large-v2]
DF 1	Microbial Stability and Sweetness Synergy Index	This feature moderately contributes to wine classification. It is strongly positively correlated with total and free sulfur dioxide and residual sugar, and strongly negatively correlated with volatile acidity and chlorides. These relationships suggest it reflects combined variations in preservation agents and acidity levels.	0.7213	0.5738	0.8387
DF 2	Non-Alcoholic Body and Mineral Content Index	This feature has the highest SHAP importance and is strongly correlated with increased density, residual sugar, and fixed acidity, while being negatively correlated with alcohol. It captures a trade-off pattern between dense, sweet, low-alcohol profiles and more alcoholic ones, which is predictive in classifying wine quality.	0.7223	0.5448	0.8157
DF 3	Acidity Structure and Freshness Index	This feature is a strong contributor to the model and is positively correlated with citric acid, fixed acidity, and alcohol, while negatively correlated with volatile acidity and pH. It represents a contrast between structured acidic-alcoholic profiles and more volatile or less acidic ones.	0.7241	0.5448	0.8517
DF 4	Oxidative Stability and Mouthfeel Balance Index	This feature has moderate importance and shows positive correlations with sulphates, pH, and free sulfur dioxide. It reflects underlying variations related to chemical stability and preservative content.	0.7693	0.6192	0.8513

TABLE XI
COMPARISON OF BASELINE CLASSIFIER AND SDFS-EXPANDED MODEL UNDER FIVE RANDOM 80/10/10 DATA SPLITS ON THE WINE QUALITY DATASET

Metric	Baseline	SDFS	Change	Interpretation
Accuracy	0.814 $\pm$ 0.009	0.853 $\pm$ 0.013	+0.039 (+4%)	SDFS helps to predict correctly more often on average.
F1-score	0.369 $\pm$ 0.034	0.620 $\pm$ 0.047	+0.25 (+67%)	SDFS helps to learn the minority class much better.
ROC-AUC	0.812 $\pm$ 0.006	0.847 $\pm$ 0.017	+0.035 (+3.5%)	Better overall separability between classes.

Note: Reported values represent the mean $\pm$ standard deviation across five independent random seeds. Bold indicates the summarized comparisons.

TABLE XII
5-FOLD CROSS-VALIDATION RESULTS ON WINE QUALITY DATASET

Metric	Baseline	SDFS	Change
Accuracy	0.826 $\pm$ 0.005	0.858 $\pm$ 0.011	+0.032 (+3.8%)
F1-score	0.399 $\pm$ 0.023	0.602 $\pm$ 0.031	+0.203 (+51%)
ROC-AUC	0.812 $\pm$ 0.010	0.869 $\pm$ 0.011	+0.057 (+7%)

Note: Average (mean $\pm$ std) results over five 80/20 train-validation splits show that the proposed SDFS model consistently improves all key metrics. Bold indicates the summarized comparisons.

model, both with and without the SDFS preprocessing, and are summarized in Table XI.

As shown in the table, the overall performance of our baseline model significantly improves when using SDFS to expand its feature space. The most notable gain is observed in the F1-score, indicating a substantial positive impact on correctly classifying minority class samples, while also achieving higher overall accuracy and AUC.

K-fold cross-validation was another technique we employed to evaluate the generalizability of our proposed model. We applied 5-fold cross-validation on the Wine Quality dataset and reported the mean $\pm$ standard deviation of various metrics for the baseline model, once on the original dataset and once after applying the SDFS model, which expanded the feature space with four additional dynamic features, as shown in Table XII. Once again, the results confirmed that SDFS enhances the model's generalization ability in classifying samples, demonstrating that the obtained improvements are both consistent and reliable.

B. Temporal-SDFS Performance

As explained earlier, we designed a separate SDFS pipeline based on an LSTM network for time-series data. To evaluate the performance of this method, we tested it on two different datasets, whose characteristics are summarized in Table II.

To evaluate the effect of SDFS preprocessing on our time-series datasets, we conducted experiments using a simple LSTM model as the final predictor network under two different scenarios. In the first scenario, we trained a baseline model using only the original samples without any feature expansion. In the second scenario, we applied the Temporal-SDFS to each dataset, introducing different numbers of dynamic features ranging from 1 to 10 to determine the optimal number of additional features that might represent previously undetected patterns in the data. The optimal number of dynamic features, along with the convergence time of the Temporal-SDFS for both datasets, is summarized in TABLE XIII. As mentioned earlier, the Temporal-SDFS uses a single initialization strategy for the dynamic features, which is random initialization, and a single method for identifying the closest trend when expanding the validation or test subsets, which is described in detail in the previous section.

As shown in the last column of Table XIII, we reported the convergence time of the Temporal-SDFS. As we can see, these values are noticeably larger than the convergence times observed for SDFS on tabular datasets. It is important to highlight that the time bottleneck in this process does not stem from the training loop itself, but rather from the Z-normalized similarity matching method used to expand the validation set in each

TABLE XIII
OPTIMAL SDFS HYPER-PARAMETERS AND CHARACTERISTICS FOR
EACH TIME-SERIES DATASET

Index	Dataset	Best Num of DF	SDFS Convergence Time (s)
1	ETTh1	3	1200
2	Air Quality UCI	2	360

TABLE XIV
COMPARISON OF BASELINE AND SDFS-EXPANDED MODELS ON TWO
TIME-SERIES REGRESSION DATASETS (ETTh1 AND AIR QUALITY)

DS	Metric	Baseline	SDFS	Change
ETTh1	MAE	0.54	0.41	-0.13 (-24%)
	RMSE	0.75	0.63	-0.12 (-16%)
	R ²	0.90	0.92	+0.02 (+2.2%)
Air Quality UCI	MAE	91.2	85.04	-6.16 (-6.7%)
	RMSE	127	115.5	-11.5 (-9%)
	R ²	0.44	0.54	+0.1 (+22.7%)

Note: Reported metrics include mean absolute error (MAE), root mean squared error (RMSE), and coefficient of determination (R^2). Lower MAE and RMSE values indicate better performance. Bold indicates the summarized comparisons.

epoch. Although this method is relatively fast compared with the other existing algorithms for comparing the shapes of two sequences, it still requires finding the closest trend within the training windows for each validation window at every epoch. This results in a computational complexity of $O(MNWF)$, where M is the number of validation windows, N is the number of training windows, W is the window size, and F is the number of static features in the dataset.

In Table XIV, we compared the performance of the baseline model with and without the Temporal-SDFS as a preprocessing step, where each dataset was expanded using the optimal number of dynamic features. As shown, applying SDFS to both datasets had a positive impact on the final mean absolute error (MAE), root mean squared error (RMSE), and coefficient of determination (R^2), demonstrating the effectiveness of our approach not only on tabular datasets and classification tasks but also on time-series prediction. Particularly, for the ETTh1 dataset, using the temporal-SDFS reduced the MAE by 24%, while for the air quality UCI dataset, which represents a more complex sequential structure, the MAE improved by up to 6%.

V. DISCUSSION

In the result section, we demonstrated a comprehensive comparison of the results extracted with and without the proposed feature expansion method and extended the analysis to feature explainability for dynamic features. In this section, we aim to discuss further the types of problem and datasets to which this approach might be more applicable, and it is more likely to lead to significant improvements.

As shown in Table VI, the Parkinson's disease detection and wine quality datasets exhibited the highest improvement due to the Accuracy metric, after applying the feature expansion technique. In contrast, the Census income dataset and Pima Indians diabetes Dataset showed a slight improvement. However, the Wisconsin breast cancer dataset even showed a decrease in

performance. Despite this, as indicated in Table IV, we achieved high accuracy and precision on the breast cancer Wisconsin dataset when feature expansion is not used.

We also included the Diabetes Health Indicators dataset as an example of a large-scale tabular dataset. Comparing the results presented in Tables IV and VI, we initially observed no change in the Accuracy metric after applying SDFS. However, a closer examination revealed that although the baseline model achieved high Accuracy, it performed poorly on other metrics, particularly Precision, because the number of diabetes cases was much smaller than nondiabetes samples. As a result, the baseline model rarely identified minority class instances correctly. After applying the SDFS preprocessing, Precision improved by up to 50%, which shows the effectiveness of the proposed approach on class-imbalanced datasets.

In general, as mentioned above and shown in TABLE III, all of these datasets are imbalanced, but they vary significantly in the number of samples, which can greatly impact the effectiveness and execution time of the feature expansion method.

Our findings suggest that our proposed feature expansion is particularly beneficial for small, imbalanced datasets where missing features can critically impact the performance whether working on tabular or time-series datasets. This is particularly true where a few crucial features are likely to be missed during the feature extraction steps and cannot be adequately addressed by a small neural network. In the study by Das et al. [31], class imbalance is recognized as a critical challenge in deep learning, often resulting in biased learning and degraded performance on minority classes. In light of this, the feature expansion technique proposed in this work may offer an effective preprocessing solution by compensating for underrepresented patterns and enhancing model generalization. SDFS serves as a complementary preprocessing step that can be selectively applied after standard normalization and cleaning procedures. In addition, it is notable that applying the feature expansion to any of our testing datasets did not cause a significant decline in the final performance, which is a positive aspect of our proposed method.

This is a particularly important observation in datasets where the original feature space is already well-explored and comprehensive, such as the Breast Cancer Wisconsin dataset, where applying feature expansion may introduce unnecessary and noisy dimensions, rather than uncovering useful or missing information. This added complexity can result in the model's biases and reduced generalization, as the newly generated features may act as random noise, ultimately leading to a slight decline in model performance.

Although the current version of SDFS was highly effective on small and imbalanced datasets, and even maintains consistent benefits on larger-scale data despite the higher computational cost, the training process can become time-consuming on very large datasets, as the dynamic feature updates are performed individually for each sample in Tabular-SDFS and for each window in Temporal-SDFS. A promising direction for future work is to explore parallelization strategies and algorithmic optimizations to accelerate the process of updating dynamic

features and trend-matching stages to improve the scalability of SDFS without compromising its accuracy or interpretability.

VI. CONCLUSION

The innovative feature engineering technique introduced in this article, SDFS is a novel method that automatically increases the dimension of the feature space. It is designed to identify potentially missing features that, if present, could positively influence the prediction of behavior in a modeled system. Using the SDFS network, we can identify missing features and approximate their corresponding values on both tabular and time-series datasets. This approach was evaluated on different open-source datasets to assess the effectiveness of this methodology in a wide range of scenarios. In addition to a considerable improvement in the performance of the models in most cases, dynamic features were also examined more closely regarding their interpretability and relevance. This analysis involved assigning a reasonable name or label to each feature based on LLM, supported by SHAP analysis. Our results demonstrate that our proposed feature expansion technique effectively enhances model performance by identifying and estimating missing features within the original feature space. This technique is particularly advantageous for small, complex datasets where conventional feature engineering methods may fail to capture critical information.

REFERENCES

- [1] J. Myung, Y. Ko, T. Kwon, J. Lee, K. Kim, and J. Song, "Intrusion detection systems based on machine learning using feature expansion methods," in *Proc. 18th Asia Joint Conf. Inf. Secur. (AsiaJCIS)*, Koganei, Japan, 2023, pp. 32–38, doi: 10.1109/AsiaJCIS60284.2023.00016.
- [2] K. Munasinghe and P. Karunanayake, "Recursive feature elimination for machine learning-based landslide prediction models," in *Proc. Int. Conf. Artif. Intell. Inf. Commun. (ICAICC)*, Jeju Island, Korea (South), 2021, pp. 126–129, doi: 10.1109/ICAICC51459.2021.9415232.
- [3] H. Al-Zoubi and S. Altaamneh, "A feature selection technique for network intrusion detection based on the chaotic crow search algorithm," in *Proc. Int. Conf. Intell. Data Sci. Technol. IEEE Int. Symp. Spread Spectr. Tech. Appl. (IDSTA)*, San Antonio, TX, USA, 2022, pp. 54–60, doi: 10.1109/IDSTA55301.2022.9923108.
- [4] U. Shaham, O. Lindenbaum, J. Svirsky, and Y. Kluger, "Deep unsupervised feature selection by discarding nuisance and correlated features," *Neural Netw.*, vol. 152, pp. 34–43, Aug. 2022, doi: 10.1016/j.neunet.2022.04.002.
- [5] Ahmad, A. F. Md Shohel, S. Khaznah, and A. Istiaque Ahmed, "Impact of missing values in machine learning: A comprehensive analysis," 2024, *arXiv:2410.08295*.
- [6] Y. Lin, Bolin Ding, H. V. Jagadish, and J. Zhou, "SMARTFEAT: Efficient feature construction through feature-level foundation model interactions," 2024, *arXiv:2309.07856*.
- [7] X. Miao, Y. Wu, L. Chen, Y. Gao, and J. Yin, "An experimental survey of missing data imputation algorithms," *IEEE Trans. Knowl. Data Eng.*, vol. 35, no. 7, pp. 6630–6650, Jul. 2023, doi: 10.1109/TKDE.2022.3186498.
- [8] X. Zhu, S. Zhang, Z. Jin, Z. Zhang, and Z. Xu, "Missing value estimation for mixed-attribute data sets," *IEEE Trans. Knowl. Data Eng.*, vol. 23, no. 1, pp. 110–121, Jan. 2011, doi: 10.1109/TKDE.2010.99.
- [9] R. Kohavi and G. H. John, "Wrappers for feature subset selection," *Artif. Intell.*, vol. 97, no. 1, pp. 273–324, 1997.
- [10] M. Dash and H. Liu, "Consistency-based search in feature selection," *J. Artif. Intell.*, vol. 151, no. 1, no. 2, pp. 155–176, 2003.
- [11] L. Yu and H. Liu, "Feature selection for high-dimensional data: A fast correlation-based filter solution," in *Proc. 20th Int. Conf. Mach. Learn. (ICML)*, 2003, pp. 856–863.
- [12] M. Moran and G. Gordon, "Curious feature selection-based clustering," *IEEE Trans. Artif. Intell.*, vol. 5, no. 12, pp. 6146–6158, Dec. 2024, doi: 10.1109/TAI.2024.3407034.
- [13] P. Mitra, C. A. Murthy, and S. K. Pal, "Unsupervised feature selection using feature similarity," *IEEE Trans. Pattern Anal. Mach. Intell.*, vol. 24, no. 3, pp. 301–312, Mar. 2002.
- [14] J. Guyon, J. Weston, S. Barnhill, and V. Vapnik, "Gene selection for cancer classification using support vector machines," *Mach. Learn.*, vol. 46, pp. 389–422, Jan. 2002.
- [15] K. D. Kantarakias, K. Shawki, and G. Papadakis, "Uncertainty quantification of sensitivities of time-average quantities in chaotic systems," *Phys. Rev. E*, vol. 101, no. 2, 2020, Art. no. 022223.
- [16] "Reconstruction, forecasting, and stability of chaotic dynamics from partial data," *Chaos: An Interdisciplinary J. Nonlin. Sci.*, vol. 30, no. 6, 2020.
- [17] J. Sun et al., "Feature expansion for graph neural networks," in *Proc. Int. Conf. Mach. Learn. PMLR*, 2023.
- [18] A. Radhakrishnan et al., "Mechanism for feature learning in neural networks and backpropagation-free machine learning models," *Science*, vol. 383, no. 6690, pp. 1461–1467, 2024.
- [19] "Missing-level statistics in classically chaotic quantum systems with symplectic symmetry," *Phys. Rev. E*, vol. 103, no. 4, 2021, Art. no. 042212.
- [20] J. Li, P. Li, Y. Zou, and X. Hu, "Multi-label learning with missing features," in *Proc. Int. Joint Conf. Neural Netw. (IJCNN)*, Shenzhen, China, 2021, pp. 1–8, doi: 10.1109/IJCNN52387.2021.9533967.
- [21] P. Cortez et al., "Modeling wine preferences by data mining from physicochemical properties," *Decis. Support Syst.*, vol. 47, pp. 547–553, Nov. 2009.
- [22] J. W. Smith, J. E. Everhart, W. C. Dickson, W. C. Knowler, and R. S. Johannes, "Using the ADAP learning algorithm to forecast the onset of diabetes mellitus," in *Proc. Annu. Symp. Comput. Appl. Med. Care*, 1988, pp. 261–265.
- [23] W. Nick et al., "Nuclear feature extraction for breast tumor diagnosis," in *Proc. Biomed. Image Process. Biomed. Vis.*, vol. 1905, SPIE, 1993.
- [24] R. Kohavi, "A study of cross-validation and bootstrap for accuracy estimation and model selection," in *Proc. 14th Int. Joint Conf. Artif. Intell. (IJCAI)*, Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 1995, pp. 1137–1143.
- [25] M. A. Little et al., "Exploiting nonlinear recurrence and fractal scaling properties for voice disorder detection," *BioMed Eng OnLine*, vol. 6, no. 23, 2007, doi:10.1186/1475-925X-6-23.
- [26] N. R. Burrows, I. Hora, L. S. Geiss, E. W. Gregg, and A. Albright, "Incidence of end-stage renal disease attributed to diabetes among persons with diagnosed diabetes — United States and Puerto Rico, 2000–2014," *MMWR Morb Mortal Wkly Rep.*, vol. 66, pp. 1165–1170, Nov. 2017, doi: 10.15585/mmwr.mm6643a2.
- [27] H. Zhou et al., "Informer: Beyond efficient transformer for long sequence time-series forecasting," in *Proc. AAAI Conf. Artif. Intell.*, vol. 35, no. 12, 2021.
- [28] S. Vito, "Air quality [Dataset]," *UCI Mach. Learn. Repository*, 2008, doi: 10.24432/C59K5F.
- [29] T. Hastie, R. Tibshirani, and J. Friedman, *The Elements of Statistical Learning: Data Mining, Inference, and Prediction*. New York, NY, USA: Springer, 2009.
- [30] D. Charte, F. Charte, M. J. del Jesus, and F. Herrera, "An analysis on the use of autoencoders for representation learning: Fundamentals, learning task case studies, explainability and challenges," *Neurocomputing*, vol. 404, pp. 93–107, 2020, doi:10.1016/j.neucom.2020.04.057.
- [31] S. Das, S. S. Mullick, and I. Zelinka, "On supervised class-imbalanced learning: An updated perspective and some key challenges," *IEEE Trans. Artif. Intell.*, vol. 3, no. 6, pp. 973–993, Dec. 2022, doi: 10.1109/TAI.2022.3160658.